



Department of Computer Science and Engineering,
Kishoreganj University

Temporary campus: Gurudayal Govt. College, Kishoreganj.

Lab Report

Course Name : **Operating System Lab**

Course Code : **CSE 3102**

Submitted by :

Name : **Md. Ehasan Mahmud**

ID : **202122104026**

Session : 2021-22

Dept. : CSE

Submitted to :

Md. Sabab Zulfiker

Lecturer & Chairman

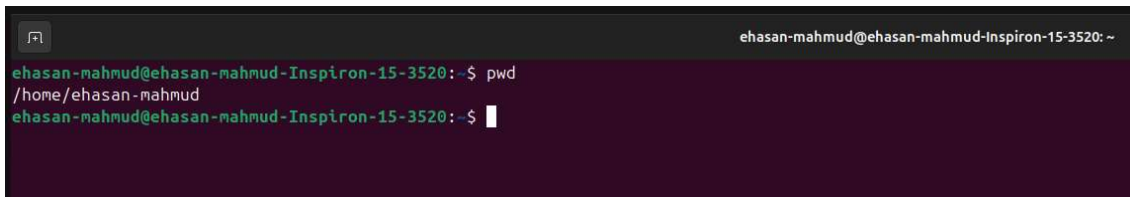
Dept. of CSE

Kishoreganj University

Command: pwd

The **pwd** (Print Working Directory) command displays the full, absolute path of the current working directory.

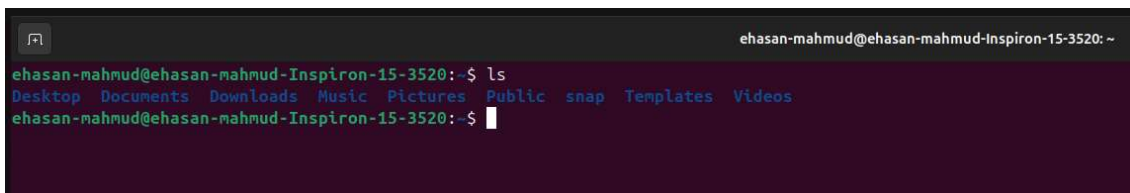
- **Purpose:** Shows the user their exact location within the file system hierarchy.
- **Usage:** pwd
- **Example (from my terminal session):**
 - pwd
 - Output: /home/ehasan-mahmud

A terminal window with a dark purple background. The title bar at the top right says "ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~". The terminal shows the command "pwd" being entered at the prompt "ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~\$". The output is "/home/ehasan-mahmud" on the next line. The prompt "ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~\$" is shown again on the third line with a cursor.

Command: ls

The **ls** (List) command is used to display the contents (files and subdirectories) of a directory.

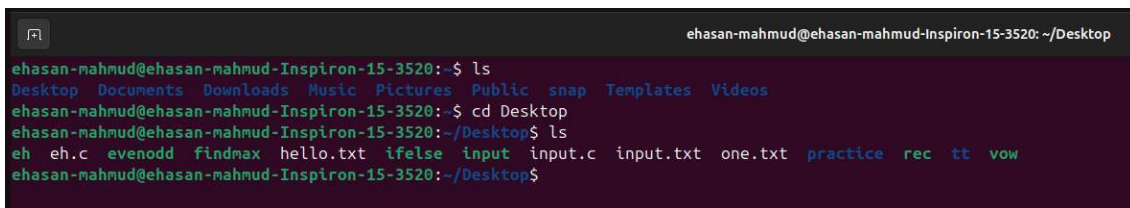
- **Purpose:** To view the contents of the current working directory or a specified directory.
- **Usage:** ls
- **Example (from my terminal session):**
 - ls
 - **Output:** *Desktop Documents Downloads Music Pictures Public snap Templates Videos*

A terminal window with a dark purple background. The title bar at the top right says "ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~". The terminal shows the command "ls" being entered at the prompt "ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~\$". The output is "Desktop Documents Downloads Music Pictures Public snap Templates Videos" on the next line. The prompt "ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~\$" is shown again on the third line with a cursor.

Command: cd

The **cd** (Change Directory) command is used to change the current working directory to a specified location.

- **Purpose:** To navigate and move between different directories within the file system hierarchy.
- **Usage:** `cd [directory_name]` or `cd [absolute_path]`
- **Example (from my terminal session):**
 - `cd Desktop`
 - **Result:** Changed the current working directory from the home directory (~) to the Desktop directory, as indicated by the prompt change: ~/Desktop.

A terminal window with a dark background and light-colored text. The prompt is 'ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop'. The user enters 'ls' and the output shows a list of directories: Desktop, Documents, Downloads, Music, Pictures, Public, snap, Templates, Videos. Then the user enters 'cd Desktop' and the prompt changes to 'ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop\$'. Finally, the user enters 'ls' and the output shows a list of files: eh, eh.c, evenodd, findmax, hello.txt, ifelse, input, input.c, input.txt, one.txt, practice, rec, tt, vow.

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~$ ls
Desktop Documents Downloads Music Pictures Public snap Templates Videos
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~$ cd Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ ls
eh eh.c evenodd findmax hello.txt ifelse input input.c input.txt one.txt practice rec tt vow
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$
```

Command: cd ..

The **cd ..** command is a specific variation of the Change Directory command used to move up one level in the directory hierarchy.

- **Purpose:** To quickly navigate from the current directory to its immediate parent directory.
- **Usage:** `cd ..`
- **Example (from my terminal session):**
 - `cd ..`
 - **Result:** Changed the current directory from /home/ehasan-mahmud/Desktop back to the user's home directory (/home/ehasan-mahmud), as confirmed by the subsequent change in the prompt from ~/Desktop back to ~

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ ls
eh eh.c evenodd findmax hello.txt ifelse input input.c input.txt one.txt practice rec tt vow
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cd ..
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~$ ls
Desktop Documents Downloads Music Pictures Public snap Templates Videos
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~$
```

Command: rm

The **rm** (Remove) command is used to delete files or directories. The essential **-r** (recursive) option is used when deleting a **directory** (folder).

- **Purpose:** To permanently remove a specified directory **and all its contents** (files and subdirectories inside it).
- **Usage:** `rm -r [directory_name]`
- **Example (from my terminal session):**
 - `rm -r evenodd`
 - **Result:** The directory named **evenodd** was deleted from the current location (`~/Desktop`), which is confirmed because the subsequent `ls` command no longer lists it.

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ ls
evenodd ifelse input practice rec tt vow
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ rm -r evenodd
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ ls
ifelse input practice rec tt vow
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$
```

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ ls
evenodd ifelse input practice rec tt vow
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ rm -r evenodd
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ ls
ifelse input practice rec tt vow
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ rm -r ifelse input practice rec tt vow
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ ls
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$
```

Note: For removing a file command is **rm [filename]**

Command: touch

The **touch** command is used to create an empty file or update the timestamp (access and modification times) of an existing file.

- **Purpose:** Primarily used for quickly creating a new, empty file or ensuring a file's timestamp is current (which is useful in scripting).
- **Usage:** touch [filename]
- **Example (from my terminal session):**
 - touch ehasan.txt
 - **Result:** A new, empty file named **ehasan.txt** was created in the current directory (~/Desktop), which is confirmed by the subsequent ls command listing the new file



```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ touch ehasan.txt
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ ls
ehasan.txt
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$
```

Command: mkdir

The **mkdir** (**M**ake **D**irectory) command is used to create new directories (folders) in the file system.

- **Purpose:** To quickly create one or more new directories in a specified location.
- **Usage:** mkdir [directory_name] or mkdir [directory1] [directory2] ...
- **Example (from my terminal session):**
 - mkdir folder1
 - mkdir folder2 folder3
 - **Result:** Three new directories named folder1, folder2, and folder3 were created in the current location (~/Desktop), as verified by the subsequent ls command.

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ mkdir folder1
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ ls
folder1
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ mkdir folder2 folder3
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ ls
folder1  folder2  folder3
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$
```

Command: nano

The **nano** command is a simple, user-friendly text editor used to create or modify files directly in the terminal.

- **Purpose:** To edit the content of a file.
- **Usage:** nano [filename]
- **Key Controls:**
 - **Ctrl + X:** Exit the editor.
 - **Y (or N):** Confirm (or decline) saving the changes before exiting.
- **Example (from my terminal session):**
 - nano ehasan.txt (Opens the file for editing.)
 - *Content added:* hi, i am ehasan.txt file.
 - *Exit sequence:* Ctrl + X, then Y.

```
GNU nano 7.2 ehasan.txt *
hi, i am ehasan.txt file.
```

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ nano ehasan.txt
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cat ehasan.txt
hi, i am ehasan.txt file.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$
```

Command: cat

The **cat** command is used to display, create, or combine the content of files directly in the terminal.

- **Purpose:** To display the content of a file, create a new file, or combine files.
- **Usage:** cat [filename]

Key Controls:

- **Ctrl + D:** Signal End-of-File (EOF) to save and exit when using the > or >> redirectors.
- **Redirection >:** Overwrites the file's content (e.g., cat > file.txt).
- **Redirection >>:** Appends new content to the end of the file (e.g., cat >> file.txt).

Example (from my terminal session):

- cat ehasan.txt (Displays the file content.)
- cat > ehasan.txt (Opens a session to **overwrite** the file content.)
- cat >> ehasan.txt (Opens a session to **append** new content to the file.)
- cat first.txt second.txt >> third.txt (Concatenates the content of two files into a third.)

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ cat ehasan.txt
hi, i am ehasan.txt file.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ cat > ehasan.txt
this is the changed writing on ehasan.txt file.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ cat ehasan.txt
this is the changed writing on ehasan.txt file.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$
```

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ cat ehasan.txt
this is the changed writing on ehasan.txt file.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ cat >> ehasan.txt
this will concatenate the writing on the ehasan.txt file.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ cat ehasan.txt
this is the changed writing on ehasan.txt file.
this will concatenate the writing on the ehasan.txt file.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$
```

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ ls
ehasan.txt first.txt second.txt third.txt third.txt
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ cat first.txt second.txt >> third.txt
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ cat third.txt
Hi, i am third.txt file.
Hi, i am first.txt file.
Hi, i am second.txt file.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$
```

Command: grep

The **grep** command (Global Regular Expression Print) is a powerful utility used to search for lines matching a specified pattern in one or more files.

- **Purpose:** To search for a specific text pattern within files and display the lines containing the match.
- **Usage:** `grep [options] "pattern" [filename]`

Key Options:

- **-i:** Ignore case (makes the search case-insensitive).
- **-n:** Display the line number of the matching lines.
- **-r:** Search recursively, including files in subdirectories (in the example, it searched multiple files in the current directory).

Example (from my terminal session):

- `grep "course" ehasan.txt`
 - **Result:** Displays the line(s) in ehasan.txt that contain the lowercase word "course" (case-sensitive match).
- `grep -i "Course" ehasan.txt`
 - **Result:** Displays the line(s) that contain "Course" or "course" or "COURSE", etc., because the **-i** option ignores case.
- `grep -ni "course" ehasan.txt`
 - **Result:** Displays the matching line, preceded by the **line number** (1:), due to the **-n** option.
- `grep -r "course"`
 - **Result:** Searches for the pattern "course" across **multiple files** (ehasan.txt and first.txt) in the current directory and prints the file name along with the matching line.


```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cat ehasan.txt
Hi, this is inside ehasan.txt file. Now i am using ubuntu for he operating system lab course.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ grep "course" ehasan.txt
Hi, this is inside ehasan.txt file. Now i am using ubuntu for he operating system lab course.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ grep -i "Course" ehasan.txt
Hi, this is inside ehasan.txt file. Now i am using ubuntu for he operating system lab course.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ grep -ni "Course" ehasan.txt
1:Hi, this is inside ehasan.txt file. Now i am using ubuntu for he operating system lab course.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$
```

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cat first.txt
Hi, its inside of first text file.
adding course for showing grep command.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ grep -r "course"
ehasan.txt:Hi, this is inside ehasan.txt file. Now i am using ubuntu for he operating system lab course.
first.txt:adding course for showing grep command.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$
```

Command: head

The **head** command is used to output the **beginning** portion of files directly to the terminal.

Purpose: To display the first few lines of a file.

Usage: head [filename]

Key Defaults:

- **Default Output:** Prints the first **10 lines** of the specified file.

Example (from my terminal session):

- head ehasan.txt (Displays the first **10 lines** of the ehasan.txt file.)

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cat ehasan.txt
its inside of ehasan text file.
line 2
line 3
line 4
line 5
line 6
line 7
line 8
line 9
line 10
line 11
line 12
line 13
line 14
line 15
line 16
line 17
line 18
line 19
line 20
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ head ehasan.txt
its inside of ehasan text file.
line 2
line 3
line 4
line 5
line 6
line 7
line 8
line 9
line 10
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$
```

Command: tail

The **tail** command is a utility used to output the **ending** portion of files directly in the terminal.

Purpose: To display the content of the **end** of a file.

Usage: tail [filename]

Key Defaults:

- **Default Output:** Prints the **last 10 lines** of the file.

Example (from my terminal session, as seen in the second image):

- cat ehasan.txt (Displays the entire 20-line file content.)
- tail ehasan.txt (Displays lines **11 through 20** of the file, which are the last 10 lines.)

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cat ehasan.txt
its inside of ehasan text file.
line 2
line 3
line 4
line 5
line 6
line 7
line 8
line 9
line 10
line 11
line 12
line 13
line 14
line 15
line 16
line 17
line 18
line 19
line 20
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ tail ehasan.txt
line 11
line 12
line 13
line 14
line 15
line 16
line 17
line 18
line 19
line 20
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$
```

C file Read:

```
GNU nano 7.2 input.c
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop

#include <stdio.h>

int main()
{
    printf("Hi, this is input.c file. I create and come inside this file using nano command.\n");
    return 0;
}
```

At first created a c file "input.c" using nano command.

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ nano input.c
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ gcc input.c -o open
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ ./open
Hi, this is input.c file. I create and come inside this file using nano command.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$
```

Command: cp

Usage: cp [source] [destination]

Demonstration: The command cp first.txt second.txt copies the contents of the file first.txt to the file second.txt.

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ cat first.txt
Hi, its inside of first text file.
adding course for showing grep command.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ cat second.txt
Hi, this is second text file.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ cp first.txt second.txt
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ cat second.txt
Hi, its inside of first text file.
adding course for showing grep command.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$
```

Command: mv

This command moves a source file to destination file. Also it is used for renaming the file.

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ ls
ehasan.txt  folder1  folder2  folder3  input.c  input.c  one.c  open  second.txt  third.txt  thired.txt
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ mv ehasan.txt second.txt
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$ ls
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop$
```

Command: chmod

chmod is a command used to change file permissions in Unix-like operating systems.

- **chmod -r second.txt:** This command removes the read permission from second.txt. As a result, when you try to cat second.txt, it says "Permission denied".
- **chmod +r second.txt:** This command adds the read permission back to second.txt. You can then successfully cat second.txt and see its content.
- **chmod -w second.txt:** This command removes the write permission from second.txt. When you try to cat > second.txt (which attempts to write to the file), it gives "Permission denied".
- **chmod +w second.txt:** This command adds the write permission back. You can then successfully write to second.txt.

- **chmod -rw second.txt:** This command removes both read and write permissions. Both `cat second.txt` and `cat > second.txt` are denied.
- **chmod +rw second.txt:** This command adds both read and write permissions back, allowing you to read and write to the file.

```

ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ chmod -r second.txt
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cat second.txt
cat: second.txt: Permission denied
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ chmod +r second.txt
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cat second.txt
its inside of second text file.

ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ chmod -w second.txt
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cat > second.txt
bash: second.txt: Permission denied
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ chmod +w second.txt
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cat > second.txt
HI, i am rewriting inside second text file.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cat second.txt
HI, i am rewriting inside second text file.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ chmod -rw second.txt
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cat second.txt
cat: second.txt: Permission denied
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cat > second.txt
bash: second.txt: Permission denied
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ chmod +rw second.txt
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cat second.txt
HI, i am rewriting inside second text file.
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$

```

Command: pgrep & kill

These commands are used for **process management** (managing running programs).

Purpose of pgrep: To find the **Process IDs (PIDs)** of running processes by their name or other attributes.

Purpose of kill: To send a **signal** (a message) to a specific process identified by its PID. The default signal is **SIGTERM (15)**, which asks the process to shut down gracefully.

From my terminal:

pgrep firefox: Finds the PID for the Firefox browser, which is **3443**.

pgrep clocks: Finds the PID for the Clocks application, which is **21475**.

pgrep calculator: Finds no PID, indicating the Calculator app was not running.

kill 21475: This command terminates the Clocks process (PID 21475), which was found using the `pgrep clocks` command.

```

ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ pgrep firefox
3443
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ pgrep clocks
21475
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ kill 21475
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ pgrep clocks
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ pgrep calculator
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$

```

Command: top

Purpose: Provides a **dynamic, real-time snapshot** of system resource usage and running processes. It is the command-line equivalent of a task manager.

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
top - 18:25:01 up 2:16, 1 user, load average: 0.47, 0.24, 0.15
Tasks: 342 total, 1 running, 339 sleeping, 2 stopped, 0 zombie
%Cpu(s): 0.3 us, 0.3 sy, 0.0 ni, 99.3 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7626.1 total, 341.0 free, 4074.2 used, 4321.4 buff/cache
MiB Swap: 4096.0 total, 4095.5 free, 0.5 used, 3551.9 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR   S   %CPU  %MEM    TIME+  COMMAND
 2279 ehasan-+  20   0 4933156 342460 157528 S   4.0   4.4   5:17.77 gnome-shell
 5565 ehasan-+  20   0 564452  62528  47656 S   2.0   0.8   0:47.33 gnome-terminal-
20132 root      0 -20     0         0     0    I   0.7   0.0   0:03.61 kworker/u49:2-i915_flip
21599 ehasan-+  20   0 14544  5904  3728 R   0.7   0.1   0:00.03 top
 104 root     -51   0     0         0     0    S   0.3   0.0   0:07.25 irq/9-acpi
 117 root      0 -20     0         0     0    I   0.3   0.0   0:00.80 kworker/4:1H-i915_cleanup
 1239 root      20   0 317196  8956  7292 S   0.3   0.1   0:03.28 upowerd
 5333 ehasan-+  20   0 2741196 295192 114220 S   0.3   3.8   0:47.17 Isolated Web Co
19322 root      20   0     0         0     0    I   0.3   0.0   0:02.57 kworker/u48:3-events_power_efficient
20383 ehasan-+  20   0 3289232 395564 213564 S   0.3   5.1   0:06.47 thunderbird-bin
   1 root      20   0  23492  13764  9156 S   0.0   0.2   0:03.52 systemd
   2 root      20   0     0         0     0    S   0.0   0.0   0:00.02 kthreadd
   3 root      20   0     0         0     0    S   0.0   0.0   0:00.00 pool_workqueue_release
   4 root      0 -20     0         0     0    I   0.0   0.0   0:00.00 kworker/R-rcu_gp
   5 root      0 -20     0         0     0    I   0.0   0.0   0:00.00 kworker/R-sync_wq
   6 root      0 -20     0         0     0    I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
   7 root      0 -20     0         0     0    I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
   8 root      0 -20     0         0     0    I   0.0   0.0   0:00.00 kworker/R-netns
  10 root      0 -20     0         0     0    I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
  13 root      0 -20     0         0     0    I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
  14 root      20   0     0         0     0    I   0.0   0.0   0:00.00 rcu_tasks_kthread
  15 root      20   0     0         0     0    I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
  16 root      20   0     0         0     0    I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
  17 root      20   0     0         0     0    S   0.0   0.0   0:00.09 ksoftirqd/0
  18 root      20   0     0         0     0    I   0.0   0.0   0:06.92 rcu_preempt
  19 root      20   0     0         0     0    S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
  20 root      20   0     0         0     0    S   0.0   0.0   0:00.01 rcu_exp_gp_kthread_worker
  21 root      rt   0     0         0     0    S   0.0   0.0   0:00.23 migration/0
  22 root     -51   0     0         0     0    S   0.0   0.0   0:00.00 idle_inject/0
  23 root      20   0     0         0     0    S   0.0   0.0   0:00.00 cpuhp/0
  24 root      20   0     0         0     0    S   0.0   0.0   0:00.00 cpuhp/2
  25 root     -51   0     0         0     0    S   0.0   0.0   0:00.00 idle_inject/2
  26 root      rt   0     0         0     0    S   0.0   0.0   0:00.33 migration/2
  27 root      20   0     0         0     0    S   0.0   0.0   0:00.03 ksoftirqd/2
  29 root      0 -20     0         0     0    I   0.0   0.0   0:00.00 kworker/2:0H-events_highpri
  30 root      20   0     0         0     0    S   0.0   0.0   0:00.00 cpuhp/4
  31 root     -51   0     0         0     0    S   0.0   0.0   0:00.00 idle_inject/4
  32 root      rt   0     0         0     0    S   0.0   0.0   0:01.80 migration/4
```

Command: ps aux | head

The command **ps aux | head** is used to display a snapshot of all currently running processes on the system and then limit that output to the first few lines. It's a quick way to see the system's overall process table and view the header and essential root processes.

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ ps aux | head
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1 23492 13764 ?        Ss   16:08   0:03 /sbin/init splash
root         2  0.0  0.0      0     0 ?        S    16:08   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        S    16:08   0:00 [pool_workqueue_release]
root         4  0.0  0.0      0     0 ?        I<   16:08   0:00 [kworker/R-rcu_gp]
root         5  0.0  0.0      0     0 ?        I<   16:08   0:00 [kworker/R-sync_wq]
root         6  0.0  0.0      0     0 ?        I<   16:08   0:00 [kworker/R-kvfree_rcu_reclaim]
root         7  0.0  0.0      0     0 ?        I<   16:08   0:00 [kworker/R-slub_flushwq]
root         8  0.0  0.0      0     0 ?        I<   16:08   0:00 [kworker/R-netns]
root        10  0.0  0.0      0     0 ?        I<   16:08   0:00 [kworker/0:0H-events_highpri]
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$
```

File Descriptor(fd)

- A **file descriptor (fd)** is a non-negative integer used by the OS to access files for **reading, writing, or closing**.
- If a file fails to open, the OS returns **-1**.
- **Flags:**
 - O_RDONLY → Read only
 - O_WRONLY → Write only
 - O_RDWR → Read & write
 - O_CREAT → Create file if not exists
 - O_APPEND → Append to existing file
- **Modes:**
 - 0644 → Owner: read/write, others: read only
 - 0666 → All: read/write
- close(fd) → Closes the file (returns -1 on failure).

Example: Creates demo.txt and writes "Hello World" using file descriptor.

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
GNU nano 7.2 input.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main()
{
    char *buf = "Hello World\n";
    ssize_t bytes_written;
    int fd = open("demo.txt", O_RDWR | O_CREAT, 0666);

    if (fd == -1) {
        printf("Error opening the file\n");
        return 1;
    }

    bytes_written = write(fd, buf, strlen(buf));
    close(fd);

    return 0;
}

ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ nano input.c
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ gcc input.c -o writer && ./writer
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ cat demo.txt
Hello World
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$
```


Fork():

- **fork()** → Used to **create a child process** under the **parent process**.
- After calling **fork()**, two processes run concurrently:
 - **Parent process** (original one)
 - **Child process** (newly created one)
- **getpid()** → Returns the **process ID (PID)** of the current process.
- **getppid()** → Returns the **parent process ID (PPID)** of the current process.

Example Output Meaning:

- Parent and child both print their own and parent IDs to show **process hierarchy**.

```
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520: ~/Desktop
GNU nano 7.2 fork.c
#include <stdio.h>
#include <fcntl.h>
#include <string.h>
#include <unistd.h>

int main() {
    printf("Starting the program...\n");
    printf("Preparing to fork a new process.\n");
    printf("Before forking\n");

    pid_t childId = fork();

    printf("After forking\n");
    printf("child id : %d\n", childId);
    printf("parent id: %d\n", getppid());
    printf("my id   : %d\n", getpid());

    return 0;
}

ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ nano fork.c
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$ gcc fork.c -o fork && ./fork
Starting the program...
Preparing to fork a new process.
Before forking
After forking
child id : 22409
parent id: 5572
my id   : 22408
After forking
child id : 0
parent id: 1993
my id   : 22409
ehasan-mahmud@ehasan-mahmud-Inspiron-15-3520:~/Desktop$
```